🛠️ Command Injection Lab: Deployment & Execution Manual

Author:Hariom M G

Target Image: docker.io/mghariom/cmd-injection-lab:v1

Difficulty: Beginner

---

1. Overview

This lab features a **Network Health Dashboard** built with Python and Flask. It contains an intentional **Command Injection** vulnerability. This manual guides you through setting up a safe environment, deploying the container, and performing the attack.

---

2. Safety & Requirements

> [!CAUTION]
> **This application is intentionally insecure.**
>
> - **DO NOT** host this on a public-facing server or cloud instance.
> - **DO NOT** run this on your primary host OS.
> - **RECOMMENDED:** Run this inside a VirtualBox or VMware instance (Ubuntu/Debian) to ensure local isolation.

System Requirements:

- **Operating System:** Linux (Ubuntu preferred) or macOS/Windows with Docker Desktop.
- **Memory:** Minimum 2GB RAM.
- **Network:** Access to Docker Hub to pull the image.

---

3. Installing Docker

If Docker is not installed on your Linux VM, run the following script:

Bash

```
# Update existing packages
sudo apt-get update

# Install prerequisite packages
sudo apt-get install -y ca-certificates curl gnupg lsb-release

# Add Docker’s official GPG key
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg

# Set up the stable repository
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Start and enable Docker
sudo systemctl start docker
sudo systemctl enable docker
```

---

4. Deploying the Lab

To start the lab, you do not need to clone any source code. You only need to pull the pre-configured image from Docker Hub.

Run the following command:

Bash

```
sudo docker run -d -p 5000:5000 --name vuln-lab mghariom/cmd-injection-lab:v1
```

Command Breakdown:

- `d`: Runs the container in the background (**Detached**).
- `p 5000:5000`: Maps host port 5000 to container port 5000.
- `-name vuln-lab`: Assigns a name to the container for easy cleanup.

---

5. Attacking the Application

1. **Access the UI:** Open a browser and navigate to `http://localhost:5000`.
2. **Verify Function:** Enter a standard IP like `8.8.8.8` to see the intended behavior.
3. **Identify the Flaw:** Use shell metacharacters (`;`, `|`, `&`) to attempt to run unauthorized commands.
   - *Example Payload:* `8.8.8.8; cat /etc/passwd`
4. **Reverse Shell (Advanced):** Set up a listener on your attacker machine (`nc -lvnp 4444`) and attempt to force the container to connect back to you.

---

6. Cleanup & Security

Once you have completed the lab, it is vital to remove the vulnerable service from your network to prevent accidental exploitation.

Bash

```
# Stop the running container
sudo docker stop vuln-lab

# Remove the container from your system
sudo docker rm vuln-lab

# (Optional) Remove the image to save disk space
sudo docker rmi mghariom/cmd-injection-lab:v1
```

---

7. Troubleshooting

- **Port 5000 is Busy:** If you get an error saying the port is already in use, check for other running containers with `docker ps` or change the port mapping to `p 8080:5000`.
- **Connection Refused:** Ensure the container status is "Up" by running `docker ps`. If it says "Exited", check the logs using `docker logs vuln-lab`.